UCA
Pontificia Universidad Católica Argentina
FACULTAD DE INGENIERÍA
Y CIENCIAS AGRARIAS

Taller: Stored Procedure

Definir que es un Stored Procedure y para que se utiliza

Bases de Datos (Relacionales)

Ing. Informática · Lic. en Cs. de Datos

Ricardo Di Pasquale
Alejandro Isidro
Edgardo Sanchez
Delfina Las Heras

AGENDA

1. Definición de Stored Procedure (SP)
2. Tablas temporales
3. Ciclo For o While
4. Trigger
5. Stored Function
6. Secuencia

Un **Stored Procedure** es, una función o un script que vive, se almacena y se ejecuta directamente dentro del motor de la base de datos

Stored Procedure

En lugar de que una aplicación externa (un backend en Python, Java, C#, etc.) envíe múltiples consultas SQL complejas a través de la red una por una, agrupamos toda esa lógica, la guardamos en la base de datos bajo un nombre específico, y simplemente la invocamos pasándole los parámetros necesarios

Stored Procedure - ejemplo

Sin Stored Procedure: Si fuera a un local de comida rápida debería decirle al cocinero: *"Corta un pan, cocina una hamburguesa, suma queso, agrega lechuga, busca papas, fríelas, y sírveme una gaseosa"*. La aplicación tiene que enviar cada instrucción (query) paso a paso a través de la red, esperando que se resuelva cada una.

Con Stored Procedure: Vas a la caja e indicas: *"Dame un Combo 1"*. La aplicación hace un simple llamado (ej. CALL GenerarCombo1()).
La cocina (la base de datos) ya tiene la "receta" predefinida, ejecuta todas las operaciones internamente de forma súper rápida, y solo te devuelve la bandeja lista (el resultado final).

¿Por que los usamos?

- **Reducción drástica del tráfico de red:** En lugar de que la aplicación envíe 50 líneas de código SQL por la red y espere resultados, solo envía el nombre del procedimiento y sus parámetros. Todo el procesamiento pesado ocurre del lado del servidor de base de datos.
- **Seguridad y Encapsulamiento (Principio de menor privilegio):** Puedes otorgarle a un usuario o aplicación el permiso para ejecutar un procedimiento llamado RegistrarNuevoAlumno(), sin necesidad de darle permisos directos de INSERT, UPDATE o DELETE sobre las tablas reales. La estructura interna de la base de datos queda oculta y protegida
- **Mantenimiento centralizado:** Si una regla de negocio cambia (por ejemplo, cambia la fórmula para calcular el impuesto de una factura), actualizas el código del procedimiento en la base de datos una sola vez. Todas las aplicaciones, reportes o microservicios que consuman esa base de datos adoptan el cambio instantáneamente sin necesidad de recompilar y desplegar código nuevo.

Ejemplo

```
CREATE OR REPLACE PROCEDURE InscribirAlumno (
   p_id_alumno IN NUMBER,
   p_id_curso  IN NUMBER
)
IS
BEGIN
   -- Paso 1: Registrar la inscripción del alumno
      INSERT INTO Inscripciones (id_alumno, id_curso, fecha_inscripcion)
      VALUES (p_id_alumno, p_id_curso, SYSDATE);

   -- Paso 2: Descontar una vacante de los cupos del curso
      UPDATE Cursos
      SET cupos_disponibles = cupos_disponibles - 1
      WHERE id_curso = p_id_curso;

   COMMIT;

END InscribirAlumno;/
```

Tablas temporales

A diferencia de SQL Server, donde la tabla temporal se crea *dentro* del Stored Procedure, en Oracle las tablas temporales se crean **una sola vez por fuera**, pero **sus datos son privados para cada sesión**.
Si el Alumno A y el Alumno B ejecutan el mismo procedimiento al mismo tiempo, ninguno verá los datos temporales del otro.

```
-- ON COMMIT DELETE ROWS indica que la tabla se vacía sola al hacer COMMIT
CREATE GLOBAL TEMPORARY TABLE Temp_Alumnos_Destacados (
    id_alumno NUMBER,
    promedio NUMBER
) ON COMMIT DELETE ROWS;
```

Tablas temporales

```
CREATE OR REPLACE PROCEDURE GenerarReporteDestacados IS
   v_total_destacados NUMBER;
BEGIN
   -- 1. Llenamos la tabla temporal con datos procesados
   INSERT INTO Temp_Alumnos_Destacados (id_alumno, promedio)
   SELECT id_alumno, AVG(nota)
   FROM Notas
   GROUP BY id_alumno
   HAVING AVG(nota) > 8;

   -- 2. Hacemos algo con esa información temporal
   SELECT COUNT(*) INTO v_total_destacados FROM Temp_Alumnos_Destacados;

   -- (Aquí podríamos insertar este resumen en otra tabla definitiva)
   DBMS_OUTPUT.PUT_LINE('Se encontraron ' || v_total_destacados || ' alumnos destacados.');

   -- 3. Al hacer COMMIT, Oracle limpia automáticamente la tabla temporal
   COMMIT;
END GenerarReporteDestacados;/
```

Ciclos FOR o WHILE

```
CREATE OR REPLACE PROCEDURE CerrarCursosVacios IS
   v_cursos_afectados NUMBER := 0; -- Iniciamos un contador en 0
BEGIN
   -- El ciclo FOR abre la consulta, lee fila por fila y se cierra solo
   FOR registro_curso IN (
      SELECT id_curso
      FROM Cursos
      WHERE cupos_disponibles = cupos_totales -- Nadie se inscribió
   )    LOOP
      -- Procesamos fila por fila usando registro_curso.id_curso
      UPDATE Cursos
      SET estado = 'CERRADO'
      WHERE id_curso = registro_curso.id_curso;

      v_cursos_afectados := v_cursos_afectados + 1;
   END LOOP;

   DBMS_OUTPUT.PUT_LINE('Se cerraron ' || v_cursos_afectados || ' cursos.');
   COMMIT;
END CerrarCursosVacios;/
```

Diferencias entre Vistas y Stored Procedure

Característica	Vista (View)	Stored Procedure (SP)
Definición	Es una consulta guardada (un SELECT). Funciona como una tabla virtual.	Es un programa guardado. Contiene lógica, variables, ciclos y múltiples acciones.
Propósito	Simplificar la lectura de datos complejos o proteger columnas sensibles.	Ejecutar acciones (Insertar, Actualizar, Borrar) y aplicar reglas de negocio.
Uso de parámetros	NO aceptan parámetros (las vistas clásicas son estáticas).	SÍ aceptan parámetros de entrada (IN) y salida (OUT).
¿Modifica datos?	Por lo general no (son de solo lectura).	Sí, esa es su función principal (hacer cambios estructurados).
Cómo se invoca	Se consulta como una tabla: SELECT * FROM Vista_Alumnos;	Se ejecuta como un comando: EXECUTE InscribirAlumno(1, 2);

UCA
Pontificia Universidad Católica Argentina

FACULTAD DE INGENIERÍA
Y CIENCIAS AGRARIAS

TRIGGER

Trigger

Un Trigger es un bloque de código (muy similar a un Stored Procedure) que vive en la base de datos, pero con una diferencia fundamental: **nadie lo invoca manualmente; se ejecuta (se "dispara") automáticamente como reacción a un evento específico en una tabla.**

Los eventos que "despiertan" a un Trigger son siempre operaciones de modificación de datos: **INSERT, UPDATE o DELETE**.

Trigger vs Stored Procedure

Stored Procedure (El Interruptor): Es como el interruptor de la luz del aula. La luz (el código) ya está instalada y lista, pero necesitas que alguien (la aplicación, el backend, el usuario) vaya y presione el botón (EXECUTE) para que se encienda. Vos controlas *cuándo* sucede.

Trigger (El Sensor de Movimiento): Es como una alarma o una luz con sensor. Nadie presiona un botón. El sistema está "escuchando" en silencio. Si alguien abre la puerta (si alguien hace un INSERT en una tabla), el sistema lo detecta y automáticamente enciende la luz o hace sonar la sirena (ejecuta el código). Es **reactivo y automático**.

3 casos de uso en el mundo real

1. **Auditoría y Bitácoras (Logs):** Se puede crear un Trigger que, cada vez que alguien haga un DELETE en la tabla Facturas, copie automáticamente los datos de esa factura a una tabla de Historial_Facturas_Borradas junto con el nombre del usuario y la fecha.
2. **Sincronización o Cálculos automáticos:** Imaginen un carrito de compras. Si alguien hace un INSERT de un nuevo producto en el detalle del carrito, un Trigger puede saltar automáticamente y hacer un UPDATE en la tabla principal del carrito para sumar el nuevo subtotal.
3. **Validaciones estrictas (Reglas de negocio complejas):** Evitar que un registro cambie de estado si no cumple ciertos requisitos. Por ejemplo, un Trigger que aborte un UPDATE si se intenta subir el sueldo de un empleado en más de un 50% de golpe.

UCA
Pontificia Universidad Católica Argentina

FACULTAD DE INGENIERÍA
Y CIENCIAS AGRARIAS

Stored Function

Stored Function

Un **Stored Function (Función Almacenada)** es un bloque de código que vive en la base de datos, estructurado de forma casi idéntica a un Stored Procedure, pero con una regla de oro inquebrantable: **está obligado a devolver un único valor** (usando la instrucción RETURN)

Mientras que un **Stored Procedure** (SP) se utiliza para ejecutar acciones que modifican el estado de la base de datos, una **Stored Function** está diseñada para realizar cálculos, transformar datos o evaluar condiciones.

Stored Function

```
CREATE OR REPLACE FUNCTION ObtenerNivelRiesgo (
   p_deuda_actual IN NUMBER
) RETURN VARCHAR2 -- ¡Importante! Aquí se define el tipo de dato que va a devolver
IS
   v_riesgo VARCHAR2(20);
BEGIN
   IF p_deuda_actual > 100000 THEN
      v_riesgo := 'ALTO';
   ELSIF p_deuda_actual BETWEEN 10000 y 100000 THEN
      v_riesgo := 'MEDIO';
   ELSE
      v_riesgo := 'BAJO';
   END IF;

   -- La regla de oro: siempre debe haber un RETURN
   RETURN v_riesgo;
END ObtenerNivelRiesgo;
/
```

Stored Function

En lugar de usar **EXECUTE**, la función se utiliza como si fuera una herramienta nativa del motor de base de datos dentro de un **SELECT**

```
SELECT
   nombre_cliente,
   deuda_actual,
   ObtenerNivelRiesgo(deuda_actual) AS clasificacion_riesgo
FROM Clientes
WHERE ObtenerNivelRiesgo(deuda_actual) = 'ALTO';/
```

Stored Function vs Stored Procedure

Característica	Stored Procedure (SP)	Stored Function
Objetivo Principal	Ejecutar lógica de negocio compleja y modificar datos.	Calcular, formatear o transformar un valor.
Devuelve un valor (RETURN)	No (puede usar parámetros OUT, pero no un RETURN de valor).	Sí, es obligatorio.
¿Modifica datos (INSERT/UPDATE/DELETE)?	Sí, es su función habitual.	No debería. (Las buenas prácticas exigen que solo hagan SELECT, para no causar efectos secundarios si se usan en consultas masivas).
Cómo se invoca	Con EXECUTE o CALL de forma independiente.	Directamente dentro de un SELECT, WHERE, o asignándolo a una variable.

UCA
Pontificia Universidad Católica Argentina

FACULTAD DE INGENIERÍA
Y CIENCIAS AGRARIAS

Secuencias

Secuencias en Oracle

Es un objeto independiente de la base de datos que genera números enteros únicos de forma secuencial.
En Oracle clásico la secuencia existe por separado y múltiples tablas o procedimientos pueden solicitarle valores

```
CREATE SEQUENCE seq_empleados
START WITH 1       -- Primer valor a generar
INCREMENT BY 1     -- De cuánto en cuánto aumenta
MAXVALUE 999999    -- Valor máximo (opcional)
NOCYCLE            -- Si llega al máximo, frena (CYCLE haría que vuelva a empezar)
CACHE 20;          -- Pre-calcula 20 valores en memoria para mayor rendimiento
```

Secuencias en Oracle

El funcionamiento de la secuencia se basa en dos pseudo-columnas fundamentales:

- NEXTVAL: Genera, guarda y devuelve el siguiente número de la secuencia. La primera vez que se llama, devuelve el valor definido en START WITH.

- CURRVAL: Devuelve el último valor generado por la secuencia en la sesión actual. Regla de oro: No se puede consultar CURRVAL en una sesión sin haber llamado a NEXTVAL al menos una vez antes.

```
CREATE OR REPLACE PROCEDURE registrar_empleado(p_nombre IN VARCHAR2)
IS
BEGIN
   INSERT INTO empleados (id_empleado, nombre)
   VALUES (seq_empleados.NEXTVAL, p_nombre);

   COMMIT;
END;
```

Secuencias en Oracle

```
CREATE OR REPLACE PROCEDURE registrar_factura(p_cliente IN VARCHAR2)
IS
   v_factura_id NUMBER;
BEGIN
   -- 1. Capturamos el ID en una variable
   v_factura_id := seq_facturas.NEXTVAL;

   -- 2. Insertamos la cabecera (Maestro)
   INSERT INTO facturas (id_factura, cliente)
   VALUES (v_factura_id, p_cliente);

   -- 3. Usamos el MISMO ID para el detalle
   INSERT INTO detalle_facturas (id_factura, item)
   VALUES (v_factura_id, 'Producto A');

   COMMIT;
END;
```

Para fijar

- Un **Stored Procedure** es un procedimiento almacenado que contiene lógica, variables, ciclos y múltiples acciones.
- Una **Vista** es una consulta guardada (un SELECT) que funciona como una tabla virtual.
- Un **trigger** es un bloque de código que se ejecuta automáticamente como reacción a un evento
- Un stored Function es un bloque de código, estructurado que está obligado a devolver un único valor
- Una secuencia es un objeto que genera números enteros únicos de forma secuencial.

UCA
Pontificia Universidad Católica Argentina

FACULTAD DE INGENIERÍA
Y CIENCIAS AGRARIAS

¡Muchas gracias! Q&A

Seguinos

@ingeagrariasuca

@IngeagrariasUCA

Ingeniería y Ciencias Agrarias

Contacto

Av. Alicia Moreau de Justo 1500
C1107AAZ, Buenos Aires, Argentina

0810-220-0822

inge_agrarias@uca.edu.ar